

Deprecate and Replace Fenv Rounding Modes

Doc. No: P2746R2

Contact: Hans Boehm (hboehm@google.com)

Audience: SG6, LEWG

Date: May 14, 2023

Target: C++26

Abstract

We argue that floating point rounding modes as specified by `fesetround()` are largely unusable, at least in C++. Furthermore, this implicit argument to floating point operations, which is almost never used, and largely opaque to the compiler, causes both language design and compilation problems. We thus make the somewhat drastic proposal to deprecate it, in spite of its long history, and to replace it with a much better-behaved facility. This is still a preliminary proposal for feedback on the shape of the replacement.

History

Issaquah, Feb. 2023, SG6

1. We would still like more feedback on use cases. So far, the only known ones are (1) bunding true results, e.g for interval arithmetic, and (2) perturbing the results to get a heuristic idea of numerical stability. Further instances of these were brought up during the meeting.
2. There was unanimous agreement that we should strive to replace `fesetround()`. There were no supporters of the current API. Most of us thought we should replace it with a library with explicit rounding mode arguments. There was some support for a C23-style statically scoped mechanism, together with some optimism that this may result in implementations. There was a volunteer to develop such a proposal. We do not plan to pursue this approach in this proposal, but welcome the opportunity to compare the two.
3. There was agreement that any library should follow the simplest possible route and not introduce a separate type for correctly rounded floats, but instead simply provide free functions on the existing types. This applies in spite of the danger of overlooking operations that could affect rounding.
4. There was also consensus that we should use nominally runtime rounding mode arguments, in spite of the valid concern that this would increase compile time. Given the scarcity of actual use cases for the current mechanism, we did not feel this is likely to turn into a significant issue.

Feb 28, 2023 C floating point committee meeting (virtual)

Also no real attachment to `fesetround()`. General preference for the C23 statically scoped `#pragma` solution. Fairly strong preference for keeping floating point exception environment dynamic. (FP exceptions were not discussed in SG6.)

Prior email discussions pointed out that the [CORE-MATH project](#) is developing libraries that (1) produce correctly rounded results for all rounding modes, and (2) generally do not actually adjust round modes. The general approach seems to be to initially carry enough extra precision for the rounding mode not to matter, and then perform one final operation that combines say, two halves of the result using the current dynamic rounding mode. This allows a C23 implementation to call a single math function to produce correctly rounded results depending on the current dynamic rounding mode. This proposal would probably remove the need for such strong guarantees, but such a library could still be used to implement correctly-rounded functions discussed here.

R1 changes: Some corrections to explanatory text. Reflected some of Jim Thomas' WG14 comments. Focussed on the free function approach favored by SG6, and outlined a more specific API.

R2 changes: Attempted to minimally remedy the BSI observation that `rint()` and friends were missing. This is probably one of the more common uses of `fenv` rounding modes.

Introduction

Currently floating point rounding modes are specified either dynamically, through the floating-point environment via `fesetround()` or, in C, statically, via `#pragma STDC FENV_ROUND`.

Searching through some large repositories of mostly nonnumerical code (e.g. on cs.android.com) we found many `fesetround()` implementation and test code, and some code to work around the existence of rounding modes, but very few real use cases. It appears to be common to encapsulate actual uses in a very small number of libraries. Earlier SG6 discussion, and the limited support in C++ (see below) also suggests that real uses are relatively rare, but more data would be helpful.

The use cases we have identified:

1. Interval arithmetic (e.g. the [CGAL library](#)) that's used to bound the true result, or other uses where it is necessary to get an accurate upper or lower bound on the true result. But such code appears to be uncommon. As we discuss below, `<fenv.h>` seems to be a particularly bad match for this. That's especially true in C++, since it inherited the facility from C, but did not keep up with recent improvements to the C standard. But even in C, its utility seems questionable.

2. As a way to specify the additional rounding mode input to the `rint()` function family. In retrospect, this seems like an (ancient) historical design mistake. The client code I managed to find generally seemed to ignore the dependency on the rounding mode, and it seems far more difficult to call these functions correctly than it would be with a rounding mode parameter.
3. As Jim Thomas points out, you could try to get some idea of floating point errors by trying with different rounding modes. (See <http://people.eecs.berkeley.edu/~wkahan/Mindless.pdf>) But, by the same argument as (3) below, that seems to be at best a little better than randomly perturbing the results. And the whole approach can break either because the program logic relies on rounding mode, or if the code itself sets the rounding mode. I believe it fails completely for $(x - y) - (y - x)$.

Although this facility appears to have very limited use, it does seem to cause a disproportionate amount of trouble, notably in the context of `constexpr` math functions. Effectively all floating point functions have an implicit rounding mode argument, set by `fesetround()`, whose value is not predictable by the compiler, confounding optimizations expected by users. In C++, it is unclear whether math libraries should respect rounding modes, or even do anything reasonable in a nonstandard rounding mode.

This approach results in many problems complicating use:

1. Implementations can't be counted on to implement it correctly. Without the `FENV_ACCESS` pragma, compilers perform optimizations that do not preserve rounding behavior. That `fenv.h` pragma is explicitly not required to be supported in C++, and hence C++ provides no guarantee that `fesetround()` behaves reasonably. (See the first note in `cfenv.syn`]).
2. In practice, it seems to be inconsistent, and somewhat unpredictable, what calls to standard math functions, and even more so, user-defined functions, do when invoked with non-standard rounding modes. (This is based on looking at some implementations. I didn't study this systematically.)
3. If you really need to control rounding to guarantee specific properties of the result, rounding modes cannot, in general, just be specified for a region of code; they need to be carefully applied to each operation. What does it mean to set the rounding mode for `cos(a + b)`? It does make sense to specify a single rounding mode if every operation in the block is monotone increasing in every argument, or for very carefully designed code (e.g. <https://hal.inria.fr/hal-03721525>.) But this is not the general case.
4. Compilers seem to commonly ignore rounding modes for compile-time evaluations, making it very hard for the programmer to predict what they are actually getting. (This is my reading anyway, though the current C standard seems to say otherwise?)
5. If we really wanted to use rounding modes to bound results, constants would also have to be rounded according to the current rounding mode. This is explicitly disallowed, even

by C, before C23: "All floating constants of the same source form(79) shall convert to the same internal format with the same value." C23 requires that static rounding modes affect constants. But this raises new issues:

- a. "Constant" macros can change meaning depending on the rounding mode context in which they are used. If the macros include arithmetic operations like subtraction or division, that change in meaning may be very different from any expectation.
 - b. The same syntactic type, like `char[(int)1.9999999999999999999]` denotes different types, depending on the rounding mode.
 - c. A "negative constant" like `-0.1` is rounded in the opposite direction of the one requested, since the minus sign is not part of the constant.
6. Especially in C++, it is unclear what the rounding modes mean for operations that are not correctly rounded to start with. Although IEEE requires correct rounding, basically no standard library implementations conform.

C's `FENV_ROUND` pragma is, in our opinion, an improvement, but not enough to actually make it very useful. Most of the above points still apply in some form.

Prior discussions

A prior email discussion of d1381r1 suggested introducing correctly rounded math functions, with an explicit rounding argument for the rare occasions on which explicit rounding modes are useful. I think that's a much better replacement, even if the set of such functions is minimal. This is the approach we pursue here.

Matthias Kretz points out that there have been prior rounding-related proposals to WG21:

* [N2899](#) Directed Rounding Arithmetic Operations (Revision 2) by G. Melquiond, S. Pion (2009-06-19) (older revisions: N2876 and N2811). This proposes free functions with explicit rounding modes, and constant suffixes, roughly along the lines we propose here. The major differences are the fact that we choose nominally run-time parameters instead of template arguments for the rounding modes.

* [P0105R1](#) Rounding and Overflow in C++ by Lawrence Crowl (2017-02-05) discusses some current narrower rounding issues, for both integers and floating point, and suggests some templated free functions to explicitly control rounding and overflow handling.

Replacement proposal

We propose to deprecate the `fesetround()` and `fegetround()` functions, moving any mention of them, and the associated Note 1 describing any use as implementation-defined, to Appendix D.

Given the long history of this facility, even in spite of its sparse use, we do not expect this to be acceptable without a replacement facility. Given the infrequent use of the current facility, our inclination is to strive for a minimalist, but extensible, replacement facility. The rest of this document focuses on such a replacement.

It is not entirely clear to us what the various rounding modes should mean in the absence of correctly rounded arithmetic. In addition, recent versions of the IEEE floating point arithmetic standard basically require correct rounding. Thus, as suggested in previous discussions, we propose to introduce a mechanism for generating correctly rounded IEEE-conforming floating point results. These would then also allow the explicit specification of rounding modes.

The design of this facility requires us to resolve several questions that do not have 100% clear answers. We make the following calls, based on SG6 discussions:

1. We provide appropriately named free functions on the existing floating point types as in N2899. This is simplest, but seems error-prone to use, in that it is easy to accidentally apply a non-correctly-rounded conversion, or even arithmetic operation as part of a computation that is intended to use e.g. a directed rounding mode. Given the low observed usage rate of the current API, it was not felt that we should complicate the proposal to address this issue.
2. N2899 suggests passing the rounding mode as a template argument. This avoids some compile time overhead of optimizing out the runtime parameter in the usual case. SG6 felt that, unlike the <atomic> API, where it is a known issue, all indications are that rounding mode APIs are very rarely used, so this should not be a major issue. Passing it as a regular argument is a bit simpler, and makes it easier to port this facility to C, should that be desired at some point.

We thus propose just the following free functions, where actual wording and syntax checking is still left as future work:

```
// Do the cr_ functions fully conform to IEC 60559?
template<floating_point F>
constexpr bool conforms_to_iec_60559();

template<floating_point F>
constexpr F cr_add(F x, F y, float_round_style r = round_to_nearest);

template<floating_point F>
constexpr F cr_subtract(F x, F y,
                        float_round_style r = round_to_nearest);

template<floating_point F>
constexpr F cr_multiply(F x, F y,
                        float_round_style r = round_to_nearest);
```

```

template<floating_point F>
constexpr F cr_divide(F x, F y,
                      float_round_style r = round_to_nearest);

// Round to a different floating point type.
// Conversion is expected to be exact if sizeof(F) >= sizeof(G)
template<floating_point F, floating_point G>
constexpr F cr_cast(G x, float_round_style r = round_to_nearest);

// Convert a string s representing a constant to the floating
// point value it represents. S may be a signed floating-point
// constant. It is implementation-defined which other
// constant strings representing constant floating-point
// expressions are supported. Throws if the argument is not supported.
template<floating_point F> constexpr F cr_const(string s);

template<floating_point F>
constexpr F cr_sqrt(F x, float_round_style r = round_to_nearest);

// Round x to an integer, according to rounding mode r,
// returning an R. R must satisfy the integral or floating_point
// concepts. Overflows handled via floating-point exceptions.
template<typename R, floating_point F>
constexpr R cr_rint(F x, float_round_style r = round_to_nearest);

// We may want to add assert_exact as another rounding mode here?

```

Currently we do not provide literals corresponding to directed rounding. This somewhat reduces the problem that -0.1 rounds in an unexpected direction.

```

    -cr_const<float>("0.1", round_toward_infinity)

```

yields -0.1 rounded downward, but hopefully makes that less surprising, but

```

    cr_const<float>("-0.1", round_toward_infinity)

```

rounds fully as expected.

We require that `cr_const()` works for normal floating point constants, optionally preceded by a minus sign. This should be easy to implement. It is in fact usually possible to guarantee correctly rounded evaluation of more complex expressions, and that would probably be ideal, though perhaps a bit challenging to specify and implement. It is not clear that this would be used enough to be worth the implementation cost, so we leave support for a wider variety of expressions implementation-defined.

Our expectation is that other math functions in the same style may eventually be added, if demand warrants. We do not add `cr_` functions until we are convinced that practically useful

implementations of the correctly rounded functions are feasible. This is already the case for many more functions than are listed here, at least for `correctly_rounded<float>`. See for example, [Lim and Nagarakatte, “High Performance Correctly Rounded Math Libraries for 32-bit Floating Point Representations”](#).

Semantics:

We propose to require that:

1. If `conforms_to_iec_60559()` yields true, then all provided `cr_` operations provide correctly rounded IEEE-conforming results.
2. At a minimum, `round_toward_infinity` and `round_toward_neg_infinity` should bound the true results.
3. In the presence of a deprecated `fesetround()`, `fesetround()` calls will not affect `cr_` operations.
4. Floating-point exceptions will be handled as they are now, i.e. via `fegetexcept()`.

A note on implementation:

For machine architectures that expect the rounding mode to be specified in floating point control register rather than in each instruction, we expect that implementations supporting `fesetround()` will need to set the control register before every sequence of `cr_` operations, and reset it to the original value when done. Normal floating point operations will continue to behave as they do now, even in the presence of `fesetround()`. If `fesetround()` is unsupported, we expect that the control register will normally indicate to-nearest rounding, and be adjusted only when a different mode is required.

Questions

Should we omit the `constexpr` and `constexpr` specifiers initially to simplify implementation? I'm leaning towards “yes”.

Is this the right way to handle constants?

Are these the right minimum semantics? Is this OK for a completely non-IEEE implementation? Do we care?

Are we missing anything for the first round?

Do we need some vector support to get around the, sometimes high, hardware overhead of switching floating point rounding modes?

Can we make floating point exception retrieval guaranteed reliable, at least for the `cr_` functions, without mandating support for the `FENV_ACCESS` pragma? Should we add variants that return a pair?